

Next, we'll further refine the ship's movement. Let's adjust the ship's speed and limit how far the ship can move so it can't disappear off the sides of the screen.

Adjusting the Ship's Speed

Currently, the ship moves one pixel per cycle through the while loop, but we can take finer control of the ship's speed by adding a `ship_speed` attribute to the `Settings` class. We'll use this attribute to determine how far to move the ship on each pass through the loop. Here's the new attribute in `settings.py`:

```
settings.py class Settings:
    """A class to store all settings for Alien Invasion."""

    def __init__(self):
        --snip--

        # Ship settings
        self.ship_speed = 1.5
```

We set the initial value of `ship_speed` to 1.5. When the ship moves now, its position is adjusted by 1.5 pixels (rather than 1 pixel) on each pass through the loop.

We're using a float for the speed setting to give us finer control of the ship's speed when we increase the tempo of the game later on. However, `rect` attributes such as `x` store only integer values, so we need to make some modifications to `Ship`:

```
ship.py class Ship:
    """A class to manage the ship."""

    def __init__(self, ai_game):
        """Initialize the ship and set its starting position."""
        self.screen = ai_game.screen
        ❶ self.settings = ai_game.settings
        --snip--

        # Start each new ship at the bottom center of the screen.
        self.rect.midbottom = self.screen_rect.midbottom

        # Store a float for the ship's exact horizontal position.
        ❷ self.x = float(self.rect.x)

        # Movement flags; start with a ship that's not moving.
        self.moving_right = False
        self.moving_left = False

    def update(self):
        """Update the ship's position based on movement flags."""
        # Update the ship's x value, not the rect.
        ❸ if self.moving_right:
            self.x += self.settings.ship_speed
```
